Lesson 6 Teacher's Key

Unity

INSTRUCTOR ONLY - do not distribute to students

Level 1 - Complete Worked Solution

A small static GameManager holding win state, called from Bullet.cs's existing trigger handler:

```
// GameManager.cs:
public static int enemiesRemaining = 5;
public static bool hasWon = false;

public static void RegisterKill() {
    enemiesRemaining--;
    if (enemiesRemaining <= 0) hasWon = true;
}

void OnGUI() {
    if (hasWon) GUI.Label(new Rect(10, 10, 200, 30), "You Win!");
}

// Bullet.cs - inside OnTriggerEnter2D:
void OnTriggerEnter2D(Collider2D other) {
    if (other.CompareTag("Enemy")) {
        AudioSource.PlayClipAtPoint(hitSound, transform.position);
        GameManager.RegisterKill();
        Destroy(other.gameObject);
        Destroy(gameObject);
    }
}
```

Level 2 - Complete Worked Solution

RegisterKill() runs exactly once per actual hit, right next to the existing Destroy(other.gameObject) call, and hasWon only ever flips from false to true - there's no path where the win text could show early, late, or more than once.

Level 3 - All Worked Solutions

Every stretch-goal idea from the handout, fully worked - not just one. Useful for validating each idea actually works before class, and as a ready reference no matter which one a student ends up picking.

Sound: play on win

Assign the provided sfx_win.wav to winSound in the Inspector, and play it the moment hasWon first becomes true, guarding against re-triggering every frame:

```
public static AudioClip winSound;

public static void RegisterKill() {
    enemiesRemaining--;
    if (enemiesRemaining <= 0 && !hasWon) {
        hasWon = true;
        AudioSource.PlayClipAtPoint(winSound,
Camera.main.transform.position);
    }
}
```

Visual: styled win text

A custom GUIStyle makes the default OnGUI text look far more intentional:

```
void OnGUI() {
    if (hasWon) {
        GUIStyle style = new GUIStyle();
        style.fontSize = 32;
        style.normal.textColor = Color.yellow;
        GUI.Label(new Rect(Screen.width/2 - 80, 20, 300, 50), "You Win!",
style);
    }
}
```

Structure: GameManager owns the counting

All win-tracking (enemiesRemaining, hasWon, RegisterKill(), OnGUI()) lives inside GameManager.cs - Bullet.cs only ever calls RegisterKill(), it never touches the counting logic or draws anything itself.

Complete Level 3 solution (all 2 goals combined)

GameManager.cs with sound and styled text combined:

```
public static int enemiesRemaining = 5;
public static bool hasWon = false;
public static AudioClip winSound;

public static void RegisterKill() {
    enemiesRemaining--;
    if (enemiesRemaining <= 0 && !hasWon) {
        hasWon = true;
        AudioSource.PlayClipAtPoint(winSound,
Camera.main.transform.position);
    }
}

void OnGUI() {
    if (hasWon) {
        GUIStyle style = new GUIStyle();
        style.fontSize = 32;
        style.normal.textColor = Color.yellow;
        GUI.Label(new Rect(Screen.width/2 - 80, 20, 300, 50), "You Win!",
style);
    }
}
```

Alternate Path Notes: Visual Scripting

A Scene Variable enemiesRemaining plus a "Decrement" node called from the Bullet's trigger graph mirrors RegisterKill().

The Level 3 goals map onto nodes the same way: an Audio node gated by a "has already played" check for sound, and an On GUI event node with styled GUI Label nodes for the visual idea.

- Keeping the same Structure principle (one place owns win-tracking) whether a student is in C# or graphs reinforces that it's a design idea, not a syntax rule.